

SHOHIN: Learned Temporal Ownership for Model-Owned Drafting, Revision, and Commitment

Anonymous Authors

Evidence snapshot: August 18, 2026

Abstract

Can a language model use additional inference computation more effectively by assigning distinct learned owners to drafting, revision, and commitment? SHOHIN tests this question without an external verifier or answer-level ensemble. On a protected 538-problem board, a dense Qwen3.5-9B draft→revision→whole-trajectory-commit system solves 383 problems versus 316 for a matched unchanged second pass. Trained revision also produces positive aggregate gains across five dense hosts spanning 0.8B–9B parameters and three families. Moving to sparse hosts, a 32,784-parameter causal temporal gate improves Qwen3.6-35B-A3B by 32/256, with positive deltas in logic, mathematics, and executable code. On non-Qwen Mixtral-8x22B, a 3.54M-parameter revision surface improves 147/1,023 to 448/1,023 and beats self-refinement by 92 answers, while exposing severe code and unchanged-case retention regressions. The combined evidence supports model-owned temporal revision as a parameter-efficient capability mechanism, but rejects universal improvement: conservative commitment remains a distinct unsolved problem at large sparse scale.

1 Introduction

Inference-time computation improves language-model performance when it is allocated through search, sampling, or adaptive refinement [10, 9]. These approaches raise a structural question: must extra compute be organized by an external verifier or answer aggregator, or can a model learn internally differentiated temporal roles that make later computation more capable than an unchanged second pass?

We study a simple hypothesis. A first model state should own a complete draft; a separately trained state should own full-trajectory revision; and a learned commitment mechanism should conservatively choose one coherent answer. This differs from asking the same model to “try again.” Prompted self-refinement can improve outputs [6], but intrinsic self-correction can also regress reasoning [5]. We therefore compare learned revision to both an unchanged second pass and matched self-refinement on the same identities, with the same model-owned draft and output budget.

Our main result is a protected dense 9B release that solves 383/538 problems, compared with 316/538 for unchanged and 374/538 for trained revision alone. The effect is not isolated to one checkpoint: aggregate revision gains repeat across Qwen3.5-0.8B, 4B, and 9B, SmolLM3-3B, and OLMo2-7B. We then test whether the temporal principle transfers to sparse models without training their native routers or experts. A causal hidden-state gate improves a 35B-total Qwen MoE, and a post-MoE low-rank revision surface produces a large replicated gain on 141B-total Mixtral.

The negative results are as important as the gains. Mixtral revision loses executable-code capability and retains only 64.6% of unchanged-correct validation identities. A learned whole-trajectory commit restores code and 93.2% retention but gives back much of the revision gain. This separates two problems that are often conflated: acquiring new capability through additional computation and preserving already-correct behavior.

Contributions.

1. A model-owned temporal architecture with separately learned draft, revision, and coherent commitment roles.
2. A protected 9B end-to-end capability result with immutable model, adapter, runtime, and result identities.
3. Aggregate revision transfer across five dense hosts and three model families.
4. Causal and cross-family MoE evidence at 35B-total and 141B-total scale, while keeping native sparse routers and experts frozen.
5. A measured capability–retention boundary that localizes the remaining large-host failure to conservative commitment.

Shohin assigns learned owners across inference time while freezing the host backbone

a Model-owned temporal lifecycle

b Sparse-host revision surface

c Tokenwise causal ownership

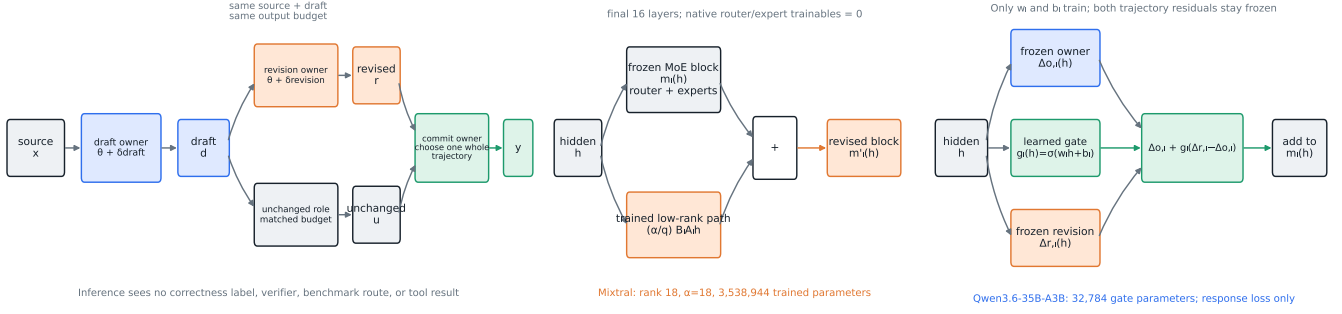


Figure 1: **Learned ownership across inference time.** (a) A model-owned draft feeds matched unchanged and trained-revision roles; commitment selects one whole trajectory. (b) The sparse-host revision surface augments the final frozen MoE blocks. (c) The causal gate interpolates frozen owner and revision residuals token by token.

2 Architecture

Let $G(\theta, p; b)$ denote bounded generation by state θ from prompt p with output budget b . The dense lifecycle first produces

$$d = G(\theta + \delta_{\text{draft}}, x; b), \quad (1)$$

then a complete revision

$$r = G(\theta + \delta_{\text{revision}}, x \| d; b). \quad (2)$$

A matched unchanged role produces a complete trajectory u from the same source and draft under the same budget. The commit owner emits one member of $\{u, r\}$; it cannot average logits or splice answer fragments.

For a frozen feed-forward or MoE block m_l at layer l , the transferable Mixtral revision surface is

$$m'_l(h) = m_l(h) + \frac{\alpha}{q} B_l A_l h, \quad (3)$$

where q is residual rank. Only A_l and B_l are trained. The executed Mixtral transfer controls the final 16 layers, uses $q = 18$ and $\alpha = 18$, and trains 3,538,944 parameters. Native routers, experts, attention, embeddings, and the language-model head remain frozen.

The Qwen causal gate freezes learned owner and revision residual branches and trains only

$$g_l(h) = \sigma(w_l h + b_l), \quad (4)$$

$$m'_l(h) = m_l(h) + \Delta_{o,l}(h) + g_l(h) [\Delta_{r,l}(h) - \Delta_{o,l}(h)]. \quad (5)$$

Thus $g_l = 0$ recovers owner behavior and $g_l = 1$ recovers revision behavior. The reported gate has 32,784 trainable parameters and uses response loss only, with no selector target or auxiliary routing label.

3 Experimental design

Every capability comparison holds evaluation identities and decoding fixed. Self-refinement receives the same draft as trained revision but uses a fixed natural-language correction instruction. The unchanged arm receives the same source, draft, and final output budget. Source-disjoint screens and holdouts are identity-separated from training inputs. Protected boards remain outside model-visible training and generation paths.

All gains are paired on exact identities. When available, we report discordant win/loss counts and exact two-sided McNemar tests. Parameter counts include only trained SHOHIN surfaces. Predeclared retention and per-domain gates remain binding even when aggregate accuracy improves, so a large numerical gain can still be a non-promotion.

The primary protected board contains GSM8K (100), MATH-500 (100), executable code (40), GPQA (198), and BBH logic (100), for 538 scored identities. The dense transfer boards contain 1,289 development or 1,279 holdout identities drawn from mathematics, logic/science, and execution-verified code. The MoE screens use a fixed source-disjoint 256-row board; Mixtral validation uses 1,023 independent rows (497 logic, 504 mathematics, 22 code).

4 Results

4.1 Protected dense release

Table 1: Protected Qwen3.5-9B result. Macro averages the five main domains; AIME is diagnostic and excluded from the 538 denominator.

System	Solved	Macro	Code	AIME
Unchanged second pass	316/538	67.263%	34/40	4/30
Trained revision	374/538	75.005%	35/40	3/30
Learned commit	383/538	75.815%	35/40	6/30
Coherent oracle (not deployable)	399/538	78.619%	35/40	6/30

The learned commit adds 67 solved problems over unchanged and nine over trained revision. An independently parameterized commit control solves 382/538, so the supported claim is useful learned coherent commitment, not a unique advantage of one antisymmetric scoring parameterization. Candidate-order consistency is 100% with zero swap error. Three prompt truncations in the product application are disclosed rather than silently recoded.

4.2 Dense transfer

Table 2: Trained revision versus matched unchanged. H and D denote holdout and development evidence.

Host	Split	Correct (revision / unchanged)	Delta
Qwen3.5-0.8B	H	328 / 242	+6.72 pp
Qwen3.5-4B	H	554 / 380	+13.61 pp
Qwen3.5-9B	H	625 / 495	+10.16 pp
SmolLM3-3B	D	469 / 358	+8.61 pp
OLMo2-7B	D	259 / 231	+2.17 pp

All five hosts improve in aggregate, but the table does not imply monotonic per-domain transfer. Qwen0.8B and SmolLM3 regress executable code; OLMo2 is too weak to promote; and the Qwen4 protected board fails its all-domain criterion.

4.3 Causal Qwen MoE transfer

On Qwen3.6-35B-A3B (35B total, 3B active), the 32,784-parameter temporal gate scores 143/256 versus 111/256 unchanged: +32 answers / +12.5 pp. It records 38 paired wins and six losses ($p = 9.43 \times 10^{-7}$), retains 105/111 unchanged-correct identities, and improves logic by 15, mathematics by 15, and code by two. Mean revision weight rises from 0.643 at the first controlled layer to 0.903 at the last, excluding a constant or degenerate gate interpretation.

4.4 Cross-family Mixtral transfer

Mixtral-8x22B has 141B total and 39B active parameters. Its revision surface is 0.00251% of total and 0.00907% of active parameters. Both draft-conditioned arms use the same immutable Qwen3.6-35B-A3B drafts, making this a deliberately cross-family model-owned transfer.

Table 3: Mixtral validation on 1,023 identities.

Arm	Correct	Logic	Math	Code
Unchanged	147	106	25	16
Self-refinement	356	224	121	11
Trained revision	448	272	176	0
Selective commit	287	222	49	16

Revision gains 301 answers over unchanged and 92 over self-refinement. The paired comparisons are 353 wins / 52 losses versus unchanged ($p = 4.44 \times 10^{-56}$) and 131 / 39 versus self-refinement ($p = 7.92 \times 10^{-13}$). The 256-row screen independently shows the same ordering: 114 revision, 105 self-refinement, and 45 unchanged.

Shohin: temporal revision transfers; commitment determines whether capability is retained

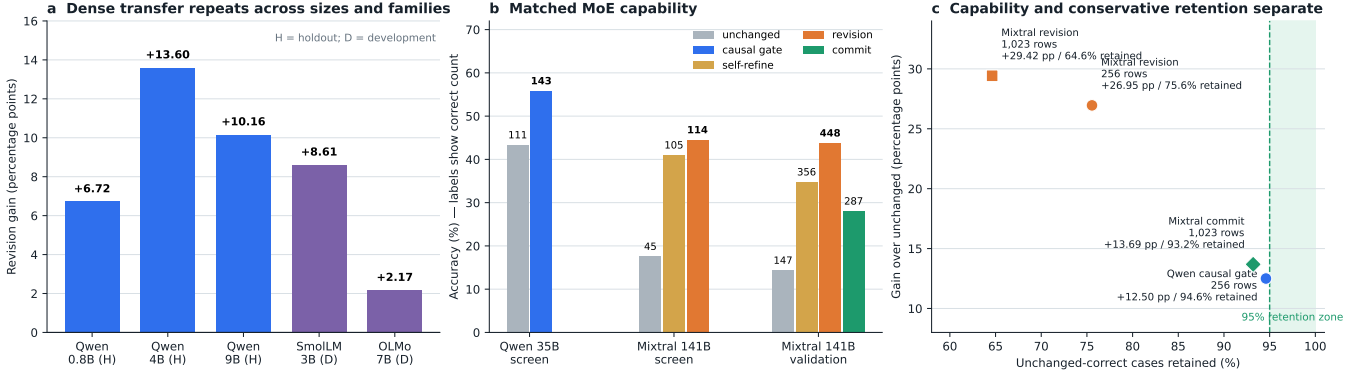


Figure 2: Capability transfer and its retention boundary. Dense gains repeat across sizes and families; both completed MoE families improve; and large aggregate gains can coexist with poor unchanged-case retention.

The result is not a qualified universal release. Revision retains only 95/147 (64.6%) unchanged-correct validation identities and loses all 22 code rows. Selective commitment restores 16/22 code and improves all domains over unchanged, but scores only 287 and retains 137/147 (93.2%), three cases below the frozen 95% floor. The formal selective-commit gate therefore fails.

An identity-joined replay of all 1,023 scored candidate triples shows that the code failure is systematic (Table 4).

Table 4: Measured Mixtral output-form collapse. Boxed is over all 1,023 rows; the final three columns are over 22 MBPP rows.

Arm	Mean tokens	Boxed	Fences	Definitions	Code correct
Unchanged	436.13	203	22	21	16
Self-refinement	296.84	454	22	19	11
Trained revision	9.99	1,023	0	0	0

Revision’s median output is nine tokens overall and seven on MBPP; no code completion contains a return statement. The learned surface has acquired an aggressive answer-extraction mode that helps answer-valued domains but violates executable output modality. Commitment restores code by selecting unmodified trajectories, localizing the next architecture problem to model-owned output-contract preservation rather than merely more reasoning capacity.

5 Discussion

Three observations survive the full evidence boundary. First, learned temporal revision is not merely an extra sample: it consistently exceeds an unchanged second pass and, on Mixtral, matched self-refinement. Second, sparse-host gains do not require training native routing or expert parameters. The causal Qwen gate learns temporal ownership over frozen residual branches, while Mixtral learns a small post-MoE residual. Third, capability and retention are distinct optimization targets. Revision finds many new correct trajectories but can overwrite already-correct behavior; commitment restores conservatism but has not yet preserved the full large-host gain.

We do not fit a monotonic scaling law from two completed MoE families. The results establish cross-family transfer at materially different active and total scales. An independently pinned GPT-OSS-120B (117B total, 5.1B active) measurement is queued as a third family point but contributes no result to this snapshot. Nemotron Super restoration failed before science, and Nemotron Ultra has not run; neither is used as evidence.

6 Related work

Self-Refine [6] and Reflexion [8] use textual feedback or reflection. SHOHIN makes self-refinement a matched control and learns a separate revision role. LoRA [4] and ReFT [11] motivate parameter-efficient weight and representation interventions; our residual surfaces add an explicit temporal contract and are evaluated with model-owned prior trajectories.

Switch Transformers [1] route tokens among experts, while Mixture-of-Depths [7] routes tokens across layer compute. Recurrent-depth latent reasoning [2] and Coconut [3] extend internal computation without requiring every step to be natural language. SHOHIN tests a complementary axis: which learned temporal state should own a token or a complete answer after a draft already exists.

7 Limitations and reproducibility

The primary protected result is one pinned 9B host and should not be read as a universal theorem. Dense aggregate gains include domain failures. The Qwen MoE evidence is a 256-row development screen. Mixtral validates at 1,023 rows but fails conservative retention and code. Public leaderboard superiority and a monotonic MoE scaling law are not supported.

All headline artifacts bind exact model revisions, trainable checkpoints, runtime manifests, source identities, arm outputs, and score reports. The protected 9B product report SHA-256 is `3e86751b...f7187`; the Qwen MoE screen is `1bd64551...b4871`; the Mixtral validation raw score is `7befd864...c3d8`. Full hashes and the complete result ledger are in the companion evidence report. Both figures are deterministically generated by `pipeline/render_shohin_publication_figure.py`.

References

- [1] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *arXiv preprint arXiv:2101.03961*, 2021.
- [2] Jonas Geiping, Sean McLeish, Neel Jain, John Kirchenbauer, Siddharth Singh, Brian R. Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, and Tom Goldstein. Scaling up test-time compute with latent reasoning: A recurrent depth approach. *arXiv preprint arXiv:2502.05171*, 2025.
- [3] Shibo Hao, Sainbayar Sukhbaatar, DiJia Su, Xian Li, Zhiting Hu, Jason Weston, and Yuandong Tian. Training large language models to reason in a continuous latent space. *arXiv preprint arXiv:2412.06769*, 2024.
- [4] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.
- [5] Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. Large language models cannot self-correct reasoning yet. In *International Conference on Learning Representations*, 2024.
- [6] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. *arXiv preprint arXiv:2303.17651*, 2023.
- [7] David Raposo, Sam Ritter, Blake Richards, Timothy Lillicrap, Peter Conway Humphreys, and Adam Santoro. Mixture-of-depths: Dynamically allocating compute in transformer-based language models. *arXiv preprint arXiv:2404.02258*, 2024.
- [8] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *arXiv preprint arXiv:2303.11366*, 2023.
- [9] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling llm test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314*, 2024.
- [10] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*, 2022.
- [11] Zhengxuan Wu, Aryaman Arora, Zheng Wang, Atticus Geiger, Dan Jurafsky, Christopher D. Manning, and Christopher Potts. Ref: Representation finetuning for language models. *arXiv preprint arXiv:2404.03592*, 2024.